Feedforward Neural Networks

Giheon Seong

Seoul National University

March 26, 2025

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

Motivation

- In a regression problem, let $\mathbf{x} \in \mathbb{R}^d$ and $y \in \mathbb{R}$ be the covariate and response, respectively.
- Many models have the form

$$y = g\left(\sum_{j=1}^{M} w_j \phi_j(\mathbf{x})\right) + \varepsilon, \tag{1}$$

  where the basis functions ϕ_j are fixed.
- In contrast, feedforward neural networks (also known as multilayer perceptrons), which are the simplest form of deep learning models, make the basis functions ϕ_j depend on parameters. This allows these parameters to be adjusted along with the coefficients $\{w_j\}$.

Motivation

- On their own, feedforward neural networks serve as the foundational building blocks for modern deep learning.

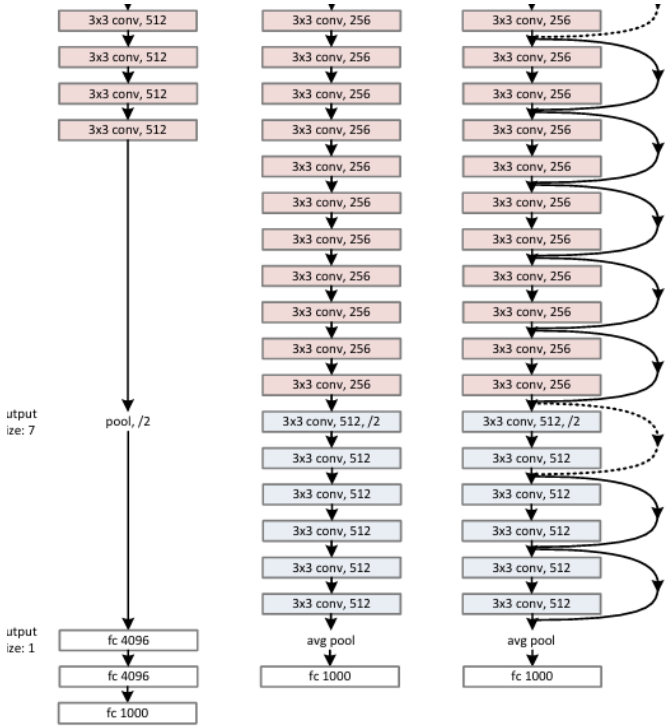

Figure 1: CNN (He et al., 2016)

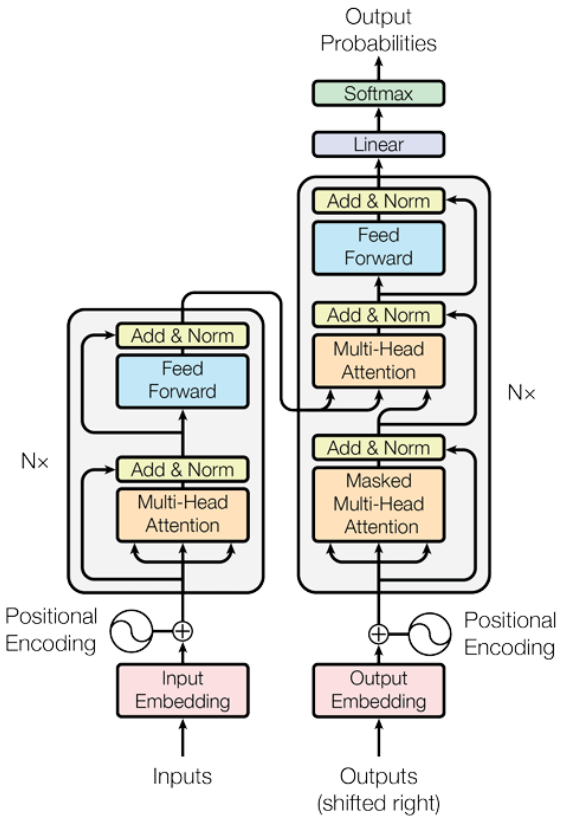

Figure 2: Transformer (Vaswani et al., 2017)

Overview

Outline of today's seminar:

- **Part 1: Approximation**
- Part 2: Optimization
- Part 3: Generalization

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

Feedforward Neural Networks

- Let $L \in \mathbb{N}$, $d_0 = d, d_1, \ldots, d_L \in \mathbb{N}$, and $\sigma : \mathbb{R} \to \mathbb{R}$ be fixed. Let $d_{L+1} := 1$.
- A feedforward neural network $\Phi : \mathbb{R}^d \to \mathbb{R}$ is the composition of a series of simple nonlinear functions:

$$\mathbf{x}^{(0)} := \mathbf{x} \in \mathbb{R}^d, \tag{2}$$

$$\mathbf{x}^{(l)} := \boldsymbol{\sigma}(\mathbf{W}^{(l-1)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l-1)}), \quad l = 1, \ldots, L, \tag{3}$$

$$\Phi(\mathbf{x}) := \mathbf{W}^{(L)}\mathbf{x}^{(L)} + \mathbf{b}^{(L)}, \tag{4}$$

for the *weight matrices* $\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l+1} \times d_l}$ and the *bias vectors* $\mathbf{b}^{(l)} \in \mathbb{R}^{d_{l+1}}$ for $l = 0, \ldots, L$. The activation function $\boldsymbol{\sigma}$ in (3) is applied entrywise.

Feedforward Neural Networks

- We refer to each $\mathbf{x}^{(l)}$ as a layer. Specifically, we refer to $\mathbf{x}^{(0)}$ as the input layer, $\mathbf{x}^{(L+1)}$ as the output layer, and all layers in between as the hidden layers.

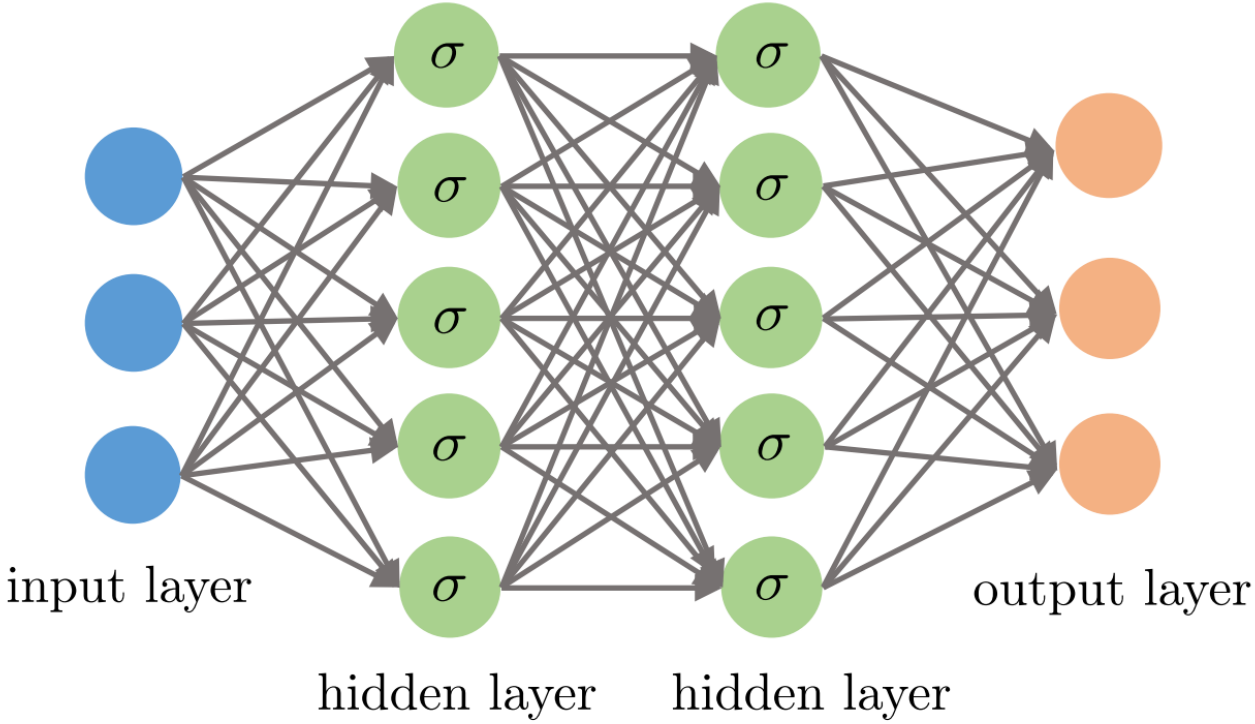

Figure 3: Feedforward Neural Networks (Fan et al., 2020)

Feedforward Neural Networks

- We call L the *depth*, $d_{max} = \max_{l=1,\ldots,L} d_l$ the *width*, σ the *activation function*, and $(\sigma; d_0, \ldots, d_{L+1})$ the *architecture* of the feedforward neural network Φ.
- If the depth is one, we call the feedforward neural network Φ *shallow*; otherwise, it is *deep*.

Activation Function

- We set σ to be nonlinear; if not, the entire feedforward neural network Φ reduces to a single affine transformation.
- Among others, typical choices for the activation function include:
  - Sigmoid: $\sigma_{\mathrm{sig}}(x) = 1/(1 + e^{-x})$,
  - ReLU (Rectified Linear Unit): $\sigma_{\mathrm{ReLU}}(x) = \max\{x, 0\}$,
  - tanh: $\sigma_{\tanh}(x) = (e^{x} - e^{-x})/(e^{x} + e^{-x})$.
- The choice of activation function can have a significant impact on performance, but there does not seem to be a universally optimal one (Petersen and Zech, 2024).

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

Error decomposition

- Let $\mathcal{F}$ be the set of all neural networks for a given architecture. We refer to $\mathcal{F}$ as the *hypothesis class*.
- Let $\mathcal{L} : \mathbb{R} \times \mathbb{R} \to [0, \infty)$ be a *loss function* and $\mathcal{D}$ be a population distribution on $\mathbb{R}^d \times \mathbb{R}$. Our goal is to find an $f \in \mathcal{F}$ that minimizes the *population risk* $\mathcal{R}$:

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[\mathcal{L}(f(\mathbf{x}), y)]. \tag{5}$$

Error decomposition

- Since $\mathcal{D}$ is typically unknown and only a sample $\{(\mathbf{x}_i, y_i)\}_{1 \le i \le n}$ is observable, we use the *empirical risk* instead:

$$\hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(\mathbf{x}_i), y_i). \tag{6}$$

- The goal of a standard training algorithm is to find

$$\operatorname{argmin}_{f \in \mathcal{F}} \hat{\mathcal{R}}_n(f) + \lambda \Omega(f) \tag{7}$$

for a regularizer Ω and $\lambda > 0$. Finding an appropriate solution for (7) is called *training*.

Error decomposition

- Let $\hat{f}_n$ be the function returned by a training algorithm, and let $f^* = \text{argmin}_f \mathcal{R}(f)$ and $f^*_{\mathcal{F}} = \text{argmin}_{f \in \mathcal{F}} \mathcal{R}(f)$.
- The excess risk is decomposed into:

$$\mathcal{R}(\hat{f}_n) - \mathcal{R}(f^*) = \underbrace{\mathcal{R}(f^*_{\mathcal{F}}) - \mathcal{R}(f^*)}_{\text{approximation error}} + \underbrace{\mathcal{R}(\hat{f}_n) - \mathcal{R}(f^*_{\mathcal{F}})}_{\text{estimation error}}. \tag{8}$$

- The approximation error does not depend on a specific estimator $\hat{f}_n$. Rather, it depends on how expressive the function space $\mathcal{F}$ is.

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

Expressive Power of Neural Networks

Universal Approximation Theorem

- A shallow network with a sufficiently large width can approximate any continuous function.

Theorem (Universal Approximation Theorem)

Let σ be a non-polynomial continuous function. Then for every $\varepsilon > 0$, every compact $K \subset \mathbb{R}^d$, and every $f \in C(\mathbb{R}^d)$, there exists a shallow network Φ such that $\sup_{\mathbf{x} \in K} |f(\mathbf{x}) - \Phi(\mathbf{x})| < \varepsilon$.

*The contents from now on heavily rely on Petersen and Zech (2024).

Expressive Power of Neural Networks

- From the perspective of the universal approximation theorem, it seems sufficient to use a shallow model.
- However, this is not the case; the theorem does not consider the number of parameters required to approximate a given function.
- Using an excessive number of parameters poses severe drawbacks regarding generalization and computational efficiency.

Expressive Power of Neural Networks

Why do we need deep networks?

- For further investigation, we fix the activation function to be σ_{ReLU}. Furthermore, we assume $d = 1$ for short to build intuition.
- A ReLU neural network is nothing but a continuous piecewise linear (CPWL) function.
- For a ReLU network to express a wide variety of complex functions, we need many pieces.

Expressive Power of Neural Networks

Theorem (Theorem 6.3 of Petersen and Zech (2024))

Let $L \in \mathbb{N}$. Then, a neural network with architecture $(\sigma_{\mathrm{ReLU}}; 1, d_1, \ldots, d_L, 1)$ has at most $(2\,\mathrm{width}(\Phi))^L$ pieces.

- While increasing the width can polynomially increase the number of pieces, increasing the depth can result in an exponential increase.
- One can deduce that for approximating complicated functions, using a deep neural network is necessary.

Expressive Power of Neural Networks

Sketch of the proof:

- Summing two CPWL functions with p_1 and p_2 pieces respectively gives a CPWL function with at most $p_1 + p_2 - 1$ pieces.
- The composition of two CPWL functions gives a CPWL function with at most $p_1 p_2$ pieces.

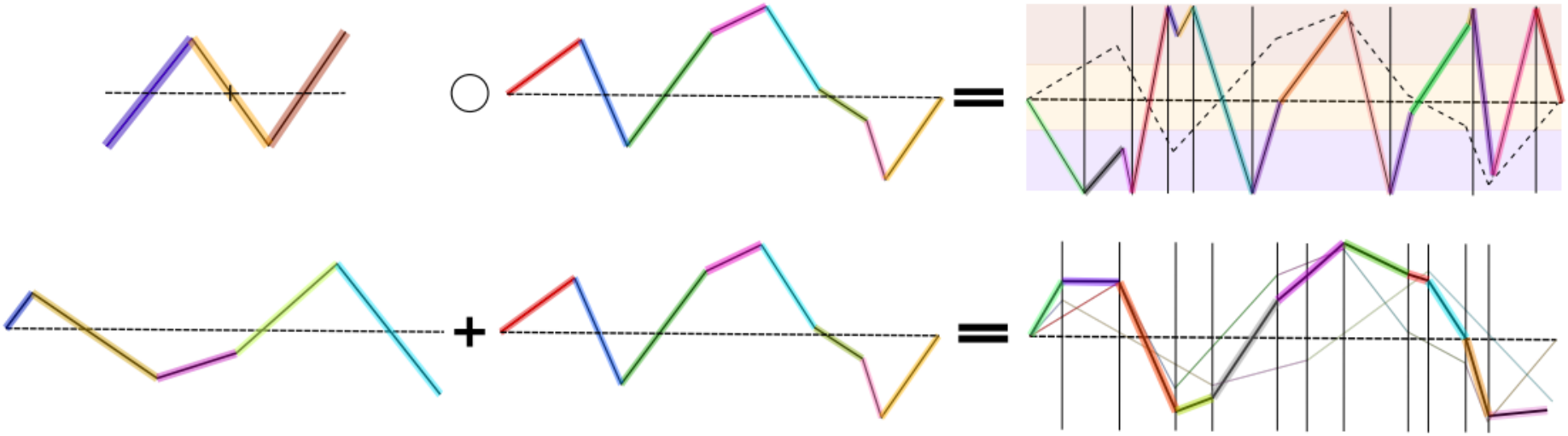

Figure 4: Composition and sum of two CPWL functions (Petersen and Zech, 2024)

Expressive Power of Neural Networks

- When $L = 1$,

$$\Phi(x) = \sum_{k=1}^{d_1} w_k^{(1)} \sigma_{\mathrm{ReLU}}(w_k^{(0)} x + b_k^{(0)}) + b^{(1)}, \quad \forall x \in \mathbb{R}.$$

This gives $\mathrm{Pieces}(\Phi) \leq 2\,\mathrm{width}(\Phi)$.

Expressive Power of Neural Networks

- When $L = 2$, we have

$$\Phi(x) = \sum_{k=1}^{d_2} w_k^{(2)} \sigma_{\mathrm{ReLU}}(h_k(x)) + b^{(2)}, \quad \forall x \in \mathbb{R},$$

  where h_k is the input to the k-th neuron of the second hidden layer.
- Thus, one can deduce that $\mathrm{Pieces}(\Phi) \leq (2\,\mathrm{width}(\Phi))^2$. We can conclude for a general L using induction.

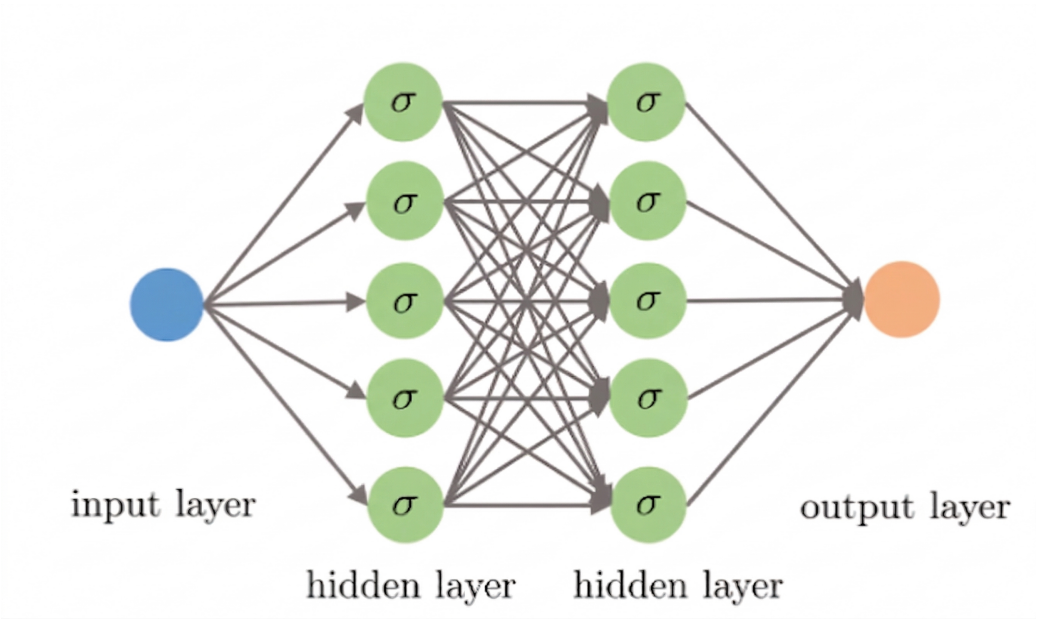

Figure 5: Deep Feedforward Neural Networks (modified from Fan et al. (2020))

Expressive Power of Neural Networks

Let size(Φ) be the total number of parameters in Φ.

Theorem (Proposition 7.6 of Petersen and Zech (2024))

Let $f : [-1, 1] \to \mathbb{R}$ be analytic but not linear. Then there exist constants $C, \beta > 0$ such that for every $\epsilon > 0$, there exists a ReLU network Φ_{deep} satisfying

$$\sup_{x \in [-1,1]} |f(x) - \Phi_{\text{deep}}(x)| \leq C \exp(-\beta \sqrt{\text{size}(\Phi_{\text{deep}})}) \leq \epsilon, \tag{9}$$

while for any shallow ReLU network Φ_{shallow}, the following holds:

$$\sup_{x \in [-1,1]} |f(x) - \Phi_{\text{deep}}(x)| \geq C^{-1} \, \text{size}(\Phi_{\text{shallow}})^{-2}. \tag{10}$$

This phenomenon is called *depth separation*.

Expressive Power of Neural Networks

- For a general d, our discussion about linear pieces extends to triangulations.
- For a general d, we have corresponding results (Montúfar et al., 2014).
- For further discussion about why deep networks are essential, refer to Chapter 3.7 of Grohs and Kutyniok (2022).

Expressive Power of Neural Networks

Approximation accuracy of deep neural networks

- Now, we turn to the problem of determining what accuracy a deep neural network can attain for a fixed size.

Definition (Definition 7.9 of Petersen and Zech (2024))

Let $k \in \mathbb{N} \cup \{0\}$, $s \in [0,1]$ and $\mathcal{X} \subset \mathbb{R}^d$. Then for $f : \mathcal{X} \to \mathbb{R}$

$$\|f\|_{C^{k,s}(\mathcal{X})} := \sup_{\boldsymbol{x} \in \mathcal{X}} \max_{\{\boldsymbol{\alpha} \in \mathbb{N}_0^d \mid |\boldsymbol{\alpha}| \leq k\}} |D^\alpha f(\boldsymbol{x})| + \sup_{\boldsymbol{x} \neq \boldsymbol{y} \in \mathcal{X}} \max_{\{\boldsymbol{\alpha} \in \mathbb{N}_0^d \mid |\boldsymbol{\alpha}| = k\}} \frac{|D^\alpha f(\boldsymbol{x}) - D^\alpha f(\boldsymbol{y})|}{\|\boldsymbol{x} - \boldsymbol{y}\|^s}.$$

Expressive Power of Neural Networks

Theorem (Theorem 7.11 of Petersen and Zech (2024))

Let $k \in \mathbb{N}_0$, $s \in [0,1]$. There exists a constant $C > 0$ such that for every $\|f\|_{C^{k,s}([0,1]^d)} < 1$ and $N \in \mathbb{N}$, there exists a ReLU network Φ_N satisfying

$$\text{size}\,(\Phi_N) = \tilde{N} \leq CN\log(N), \quad \text{depth}\,(\Phi_N) \leq C\log(N), \tag{11}$$

and

$$\sup_{\mathbf{x}\in[0,1]^d} |f(\boldsymbol{x}) - \Phi_N(\boldsymbol{x})| \leq CN^{-\frac{k+s}{d}}. \tag{12}$$

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

Curse of Dimensionality

- To attain a given accuracy $\varepsilon > 0$, we need a neural network with a size exponential to the dimension, which is undesirable:

$$N = O(\varepsilon^{-\frac{d}{k+s}}). \tag{13}$$

- We will assume that f is in a restricted function class with an implicit dimension m. Then, (13) turns into:

$$N = O(\varepsilon^{-\frac{m}{k+s}}). \tag{14}$$

- This function class includes:
  - Functions on an m-dimensional manifold.
  - Functions with a compositionality structure.

Curse of Dimensionality

Functions on an m-dimensional manifold

Theorem (Proposition 8.18 of Petersen and Zech (2024))

Let $k \in \mathbb{N}_0$ and $s \in [0,1]$, and let $\mathcal{M}$ be a smooth, compact m-dimensional manifold in $\mathbb{R}^d$. Then there exists a constant $C > 0$ such that for all $\|f\|_{C^{k,s}(\mathcal{M})} < 1$ and $N \in \mathbb{N}$, there exists a ReLU network Φ_N such that

$$\text{size}(\Phi_N) \leq CN\log(N), \quad \text{depth}(\Phi_N) \leq C\log(N), \tag{15}$$

and

$$\sup_{\mathbf{x}\in\mathcal{M}} |f(\mathbf{x}) - \Phi_N^f(\mathbf{x})| \leq C\|f\|_{C^{k,s}(\mathcal{M})} N^{-\frac{k+s}{m}}. \tag{16}$$

Curse of Dimensionality

Functions with a compositionality structure

Now, we assume that high-dimensional functions are constructed by composing many low-dimensional functions. Consider a directed acyclic graph $\mathcal{G}$ with M vertices $\eta_1, \ldots, \eta_M$ such that:

- exactly d vertices, $\eta_1, \ldots, \eta_d$, have no incoming edges,
- each vertex has at most $m \in \mathbb{N}$ incoming edges,
- exactly one vertex, η_M, has no outgoing edges.

Curse of Dimensionality

- Let S_j be the set of indices i where we have an edge from η_i to η_j.
- We let $F_j = x_j$ for all $j \leq d$ and $F_j = f_j((F_i)_{i \in S_j})$ for all $j > d$.

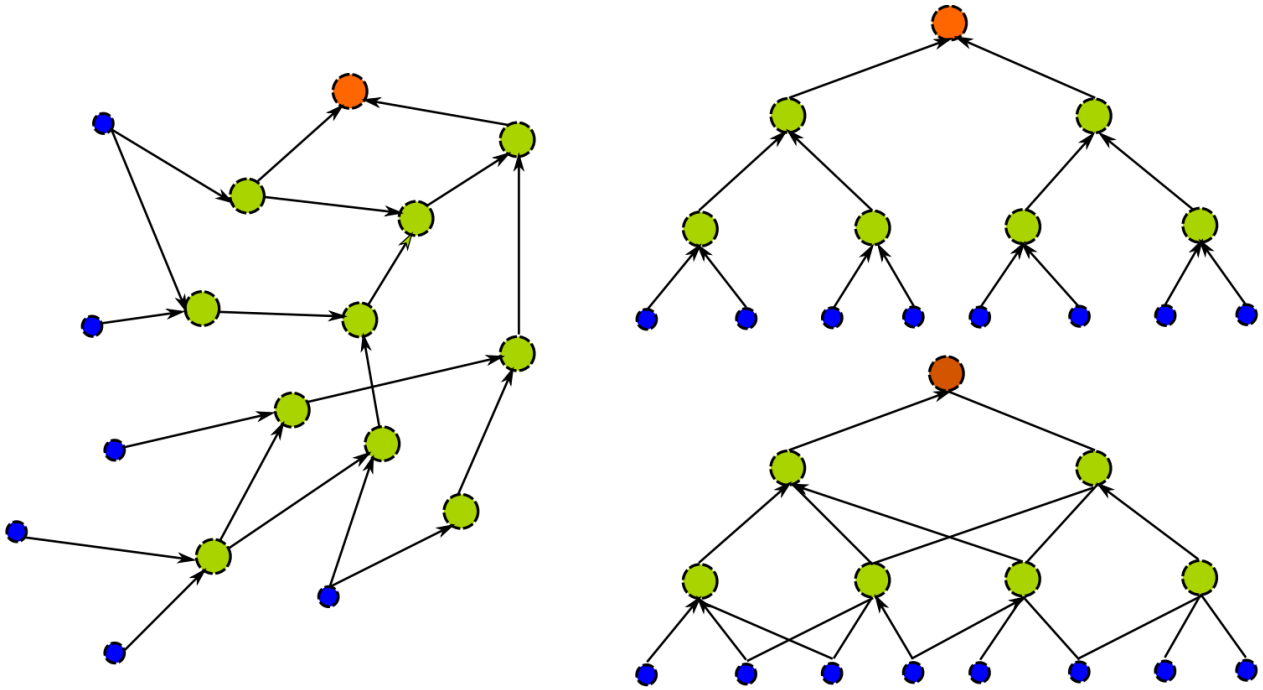

Figure 6: Examples of $\mathcal{G}$ with at most 3 incoming edges (Petersen and Zech, 2024).

Curse of Dimensionality

Letting $d_j = |S_j|$, f_j is a function from $\mathbb{R}^{d_j}$ to $\mathbb{R}$.

Theorem (Proposition 8.16 of Petersen and Zech (2024))

Let $k, m, d, M \in \mathbb{N}$ and $s > 0$. For the function F_M defined before, assume $\|f_j\|_{C^{k,s}(\mathbb{R}^{d_j})} < 1$ for all $j > d$. Then there exists a constant $C = C(m, k+s, M)$ such that for every $N \in \mathbb{N}$, there exists a ReLU network Φ_N satisfying

$$\text{size}\,(\Phi_N) \leq CN\log(N), \quad \text{depth}\,(\Phi_N) \leq C\log(N)$$

and

$$\sup_{x\in[0,1]^d} |F_M(x) - \Phi_N(x)| \leq N^{-\frac{k+s}{m}}.$$

Curse of Dimensionality

- Applications of this framework include an important class in nonparametric statistics: generalized additive models

$$y = F(m_1(X_1) + \cdots + m_d(X_d)) + \epsilon, \tag{17}$$

  where $F, m_1, \ldots, m_d$ are unknown.
- In this case, it corresponds to $m = 1$ and $M = 2d + 1$ in our framework.
- For further details, refer to Schmidt-Hieber (2020) and Kohler and Langer (2021).

Table of Contents

1. Introduction
2. Basic Setup
3. Error decomposition
4. Expressive Power of Neural Networks
5. Curse of Dimensionality
6. Reference

References I

Fan, J., Ma, C., and Zhong, Y. (2020). A selective overview of deep learning. *Statistical science: a review journal of the Institute of Mathematical Statistics*, 36(2):264.

Grohs, P. and Kutyniok, G. (2022). *Mathematical aspects of deep learning*. Cambridge University Press.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.

Kohler, M. and Langer, S. (2021). On the rate of convergence of fully connected deep neural network regression estimates. *The Annals of Statistics*, 49(4):2231–2249.

Montúfar, G., Pascanu, R., Cho, K., and Bengio, Y. (2014). On the number of linear regions of deep neural networks. *Advances in neural information processing systems*, 27.

References II

Petersen, P. and Zech, J. (2024). Mathematical theory of deep learning. *arXiv preprint arXiv:2407.18384*.

Schmidt-Hieber, J. (2020). Nonparametric regression using deep neural networks with relu activation function. *The Annals of Statistics*, 48(4):1875.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Thank You